



Set de Práctica 0

Objetivos

- Diseñar algoritmos simples en C.
- Comprender las diferencias de uso entre Arrays y Listas Ligadas.
- Familiarizarse con el uso de punteros y manejo de memoria.

Definición de lista ligada simple (LibEDD)

En términos generales, una lista ligada simple (singly linked list, abreviado SLL) se compone de una serie de nodos que contienen dos elementos: el dato que guardan, y una referencia al siguiente nodo (en C, esto sería un puntero).

Para este set de práctica, nos centraremos en el uso y funcionamiento de esta estructura de datos. Además, *al igual que en los talleres que utilicen listas ligadas simples*, la implementación en C que usarán de SLLs está provista por LibEDD, por lo que, si quieren consultar los detalles de implementación, pueden revisar las guías de talleres del curso y el archivo `libedd/libedd_sll.h` que se encuentra en este set.

De todos modos, para poder realizar los ejercicios, sólo necesitan saber cómo están definidos los `structs` con los que se modela la estructura de datos en el código, que se muestran a continuación:

`libedd_sll.h`

```
1 typedef struct sll_node {
2     int data;
3     struct sll_node *next;
4 } SllNode;
5
6 typedef struct sll {
7     SllNode *head;
8     SllNode *tail;
9     size_t size;
10 } Sll;
```

Como podemos ver, el `struct SllNode` contiene un número entero `data`, que representa la información que contiene cada nodo de la lista, y un puntero a otro `SllNode` llamado `next`, que representa la referencia al siguiente valor de la lista. Por otro lado, el `struct` que define la lista ligada como tal es `Sll`, que tiene tres elementos: un puntero a un `SllNode` que representa la cabeza de la lista (el primer elemento), un puntero a un `SllNode` que representa la cola de la lista (el último elemento), y por último un número de tipo `size_t` (tipo utilizado para indicar el tamaño de algo, siempre positivo) llamado `size` que indica el tamaño de la lista. Un punto importante es que la solución desarrollada a cada ejercicio debería mantener estos valores actualizados correctamente, ya que otras funciones que utilicen una SLL podrían depender de ello.

Ejercicios

ENTER {K}

Deberás encargarte de insertar el número K en la lista ligada, esto es, que quede después del último elemento existente.

Tras insertar el número deberás mostrar el siguiente output:

output	
1	Se ha insertado el numero {K} en la lista ligada.

DELETE {K}

Deberás encargarte de buscar el número K dentro de la lista ligada.

1. En caso de encontrarlo deberás eliminarlo y mostrar el siguiente output:

output	
1	Se ha eliminado el numero {K} en la lista ligada.

2. En caso de no encontrar el número deberás mostrar el siguiente output:

output	
1	No se ha encontrado el numero {K} en la lista ligada.

REVERSE

Deberás encargarte de invertir la lista ligada, esto es, el primer elemento se convierte en el último, el segundo elemento en el penúltimo, y así hasta llegar al último, que se convertirá en el primer elemento de la lista ligada.

Una vez invertida, deberás mostrar todos sus elementos:

output	
1	Se ha invertido la lista ligada:
2	{elemento_0}
3	{elemento_1}
4	.
5	.
6	.
7	{elemento_n}

SHIFT {K}

Deberás encargarte de mover cada elemento de la lista ligada K posiciones hacia la derecha. En el caso de tener que mover el último elemento, este pasa al inicio de la lista ligada. Por ejemplo, si se tiene la lista ligada 1-3-4 y hacemos SHIFT con K = 1, pasan a estar en el orden 4-1-3, y si hicieramos SHIFT con K = 2, pasan a estar en el orden 3-4-1.

Una vez hecho el shift, deberás mostrar todos sus elementos:

output	
1	Se ha hecho el shift de {K} posiciones en la lista ligada:
2	{elemento_0}
3	{elemento_1}
4	.
5	.
6	.
7	{elemento_n}

SPLIT

Deberás encargarte de separar la lista ligada en dos, de la siguiente forma:

- La primera lista ligada tendrá todos los elementos que estén en posiciones impares.
- La segunda lista ligada tendrá todos los elementos que estén en posiciones pares.

Por ejemplo, si tenemos la lista ligada 2-1-4-3, tras hacerle split:

- La primera lista ligada tendrá los elementos en posiciones impares, por lo que se verá así: 1-3.
- La segunda lista ligada tendrá los elementos en posiciones pares, por lo que se verá así: 2-4.

Una vez realizado el split, deberás mostrar todos los elementos de ambas listas ligadas:

```
output
1 | Se ha realizado el split de la lista ligada.
2 | Primera lista resultante:
3 | {elemento_0}
4 | {elemento_1}
5 | .
6 | .
7 | .
8 | {elemento_n}
9 | Segunda lista resultante:
10| {elemento_0}
11| {elemento_1}
12| .
13| .
14| .
15| {elemento_m}
```

Ejecución

Tu programa se debe compilar con el comando **make** (puedes limpiar el output de compilación con **make clean**) y debe generar un ejecutable de nombre **set-0** que se ejecuta con el siguiente comando:

```
./set-0 INPUT_PATH OUTPUT_PATH
```

donde **INPUT_PATH** será la ruta a un archivo con los eventos a simular y **OUTPUT_PATH** la ruta del archivo donde se guardarán los resultados. Por ejemplo:

```
./set-0 tests/input/test_01.txt outputs/test_01.txt
```

Si quieres verificar que el output generado por tu código es igual al esperado en un test (es decir, tu código pasa el test), puedes hacerlo con el siguiente comando:

```
diff EXPECTED_OUTPUT_PATH PROGRAM_OUTPUT_PATH
```

donde **EXPECTED_OUTPUT_PATH** será la ruta a un archivo con el output esperado del test, y **PROGRAM_OUTPUT_PATH** la ruta del archivo con el output de tu programa para dicho test. Si este comando finaliza sin imprimir nada en consola, significa que los archivos que entregaste son idénticos, y por tanto que el test pasa correctamente. En caso contrario, lo que se imprime es la diferencia entre los dos archivos, lo que podría ayudarte a ver porqué el test falló.

En caso de querer correr el programa para ver los leaks y errores de memoria utilizando **valgrind**, deberás utilizar el siguiente comando:

```
valgrind ./set-0 INPUT_PATH OUTPUT_PATH
```

A continuación detallamos los archivos de input y output.

Input

El archivo de input comenzará con el número **E** que corresponde al número de eventos a recibir, y las siguientes **E** líneas corresponden a cada uno de estos eventos.

Como ejemplo tenemos el siguiente input:

input	
1	9 # Cantidad de eventos
2	ENTER 1
3	ENTER 4
4	ENTER 7
5	DELETE 7
6	ENTER 6
7	ENTER 9
8	REVERSE
9	SHIFT 2
10	SPLIT

Output

output	
1	Se ha insertado el numero 1 en la lista ligada.
2	Se ha insertado el numero 4 en la lista ligada.
3	Se ha insertado el numero 7 en la lista ligada.
4	Se ha eliminado el numero 7 en la lista ligada.
5	Se ha insertado el numero 6 en la lista ligada.
6	Se ha insertado el numero 9 en la lista ligada.
7	Se ha invertido la lista ligada:
8	9
9	6
10	4
11	1
12	Se ha hecho el shift de 2 posiciones en la lista ligada:
13	4
14	1
15	9
16	6
17	Se ha realizado el split de la lista ligada.
18	Primera lista resultante:
19	1
20	6
21	Segunda lista resultante:
22	4
23	9